

PROGRAMACIÓN ESTRUCTURADA

FICHA TÉCNICA DE LA UNIDAD

Unidad Trabajo	UT2. Programación estructurada
Resultados de Aprendizaje	RA2. Maneja los elementos necesarios en un lenguaje de programación, identificando los bloques fundamentales de construcción de un programa.
Criterios de Evaluación	CE2.a: Manejar los tipos de datos básicos (numéricos, booleanos, cadenas) y sus operaciones. CE2.b: Utilizar estructuras de control de flujo (condicionales y bucles) para resolver problemas algorítmicos. CE2.c: Implementar la entrada y salida de datos estándar de forma eficiente y legible. CE2.d: Documentar el código mediante comentarios adecuados para facilitar su mantenimiento.
Tiempo	15 horas

ÍNDICE DE CONTENIDOS

[1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN](#)

[2. FUNDAMENTOS DE PYTHON](#)

[2.1. El Concepto de Script frente a Programa Ejecutable](#)

[2.2. Variables, Ámbitos y Asignación Dinámica](#)

[2.3. Literales y Comentarios al Código](#)

[3. TIPOS DE DATOS PRIMITIVOS E IDENTIFICACIÓN](#)

[3.1. Datos Numéricos: Enteros \(int\) y Reales \(float\).](#)

[3.2. Operadores Aritméticos de Sistemas](#)

[3.3. Funciones Numéricas Predefinidas](#)

[3.4. Datos Booleanos y Operaciones Lógicas/Relacionales](#)

[3.5. Datos de Texto: Cadenas de Caracteres \(strings\).](#)

[4. ENTRADA Y SALIDA ESTÁNDAR](#)

[4.1. Captura de Datos con input\(\) y Conversión de Tipos \(Casteo\)](#)

[4.2. Visualización con print\(\).y Formateo Básico](#)[5. ESTRUCTURAS DE CONTROL DE FLUJO LÓGICO](#)[5.1. La Regla Sintáctica de la Indentación](#)[5.2. Alternativa Condicional \(if - elif - else\)](#)[5.3. Bucle Iterativo Condicional \(while\).y Controladores \(break\).](#)

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El aprendizaje de la programación estructurada constituye el primer paso metodológico real para la cualificación de un Administrador de Sistemas hacia el ámbito de la automatización (*scripting*) y la filosofía DevOps. Esta metodología se basa en un principio fundamental: **las instrucciones de un script se ejecutan de manera lineal y secuencial**, una detrás de otra, salvo cuando alteramos deliberadamente este orden natural mediante el uso de decisiones condicionales o repeticiones de código (bucles).

Relación Curricular:

Esta unidad didáctica se fundamenta de forma directa sobre la lógica algorítmica y el pensamiento computacional adquiridos en la unidad anterior utilizando pseudocódigo en PSeInt (UT1). A nivel interdisciplinar, se conecta con:

- **Administración de Sistemas Operativos (2º ASIR):** Al sentar las bases sintácticas necesarias para sustituir los antiguos scripts en Bash por scripts avanzados de Python capaces de realizar comprobaciones básicas de uso de hardware y control de credenciales de usuario.
- **Gestión de Bases de Datos (1º ASIR):** Al capacitar al alumno en la comprensión del uso de operadores lógicos y relacionales, fundamentales para comprender el filtrado y estructuración de información en sistemas de bases de datos.

2. FUNDAMENTOS DE PYTHON

2.1. El Concepto de Script frente a Programa Ejecutable

A diferencia de los lenguajes compilados de bajo o alto nivel (como C o C++), que traducen todo su código a un archivo binario ejecutable antes de correr en la máquina, **Python es un lenguaje de alto nivel interpretado**.

Esto significa que las instrucciones se guardan en un archivo de texto plano con la extensión .py. Al ejecutarlo, un programa intermedio denominado **intérprete** lee y procesa el código línea a línea en tiempo de ejecución.

- **Ventaja para el SysAdmin:** No hay fase previa de compilación. Puedes escribir tu código e implementarlo de manera inmediata en servidores con distintos sistemas

operativos (Linux, Windows o macOS) siempre que tengan instalado el intérprete de Python.

2.2. Variables, Ámbitos y Asignación Dinámica

Una **variable** es una referencia lógica o un nombre que apunta a un espacio en la memoria RAM del servidor para guardar un dato que puede cambiar durante la ejecución del script.

Para guardar un dato dentro de una variable, utilizamos el operador de asignación `=`. Python utiliza un mecanismo denominado **tipado dinámico**: no requiere que declares de qué tipo es la variable antes de usarla, sino que el propio intérprete detecta el tipo de dato de forma automática en el momento en que le asignas un valor.

- **Ámbito de una variable (Scope):** Hace referencia a las partes del programa donde una variable existe y se puede leer o escribir.
 - **Variables Globales:** Se definen en el flujo principal de nuestro script y son accesibles desde cualquier parte de nuestro código actual.
 - **Variables Locales:** Se definen dentro de una estructura específica (como una función o bloque de control) y solo existen mientras dure la ejecución de ese bloque.

2.3. Literales y Comentarios al Código

- **Literales:** Son valores fijos que se escriben directamente en el código del programa. Por ejemplo, en la expresión `umbral = 90`, el número 90 es un literal numérico entero.
- **Comentarios:** Son anotaciones en el código que el intérprete de Python ignora por completo durante la ejecución. En administración de sistemas, es obligatorio documentar qué hace cada script o línea compleja. Los comentarios de una sola línea se inician siempre con el carácter almohadilla `#`.

3. TIPOS DE DATOS PRIMITIVOS E IDENTIFICACIÓN

3.1. Datos Numéricos: Enteros (int) y Reales (float)

Python trabaja de manera nativa con dos tipos de datos numéricos principales:

- **Enteros (int):** Representan números sin parte decimal, ya sean positivos, negativos o el cero.
- **Reales o de punto flotante (float):** Representan números con coma o decimales (se escribe con punto `.`).

Podemos utilizar la función integrada `type()` para averiguar en cualquier momento la clase de dato que almacena una variable.

A) Ejemplo Básico:

```
# Definición de dos tipos de números simples
edad_usuario = 25
altura_metros = 1.78
print(type(edad_usuario)) # Imprime: <class 'int'>
print(type(altura_metros)) # Imprime: <class 'float'>
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: inventario_servidor.py
# Guardamos valores para monitorizar el estado físico de un nodo
servidores_en_rack = 14 # Tipo 'int': Unidades enteras
indivisibles
consumo_electrico_amperios = 4.85 # Tipo 'float': Medición precisa
con decimales
print("Tipo de la variable servidores_en_rack:")
print(type(servidores_en_rack))
print("Tipo de la variable consumo_electrico_amperios:")
print(type(consumo_electrico_amperios))
```

3.2. Operadores Aritméticos de Sistemas

Símbolos matemáticos utilizados para manipular los valores numéricos:

Operador	Operación	Ejemplo	Resultado
+	Suma	5 + 2	7
-	Resta	5 - 2	3
*	Multiplicación	5 * 2	10
/	División (siempre devuelve decimal)	5 / 2	2.5
//	División entera (elimina los decimales)	5 // 2	2
%	Módulo (devuelve el resto de la división)	5 % 2	1
**	Potencia	5 ** 2	25

(Nota: Los operadores unarios + y - actúan para representar la polaridad positiva o negativa de un valor numérico).

A) Ejemplo Básico:

```
# Operaciones matemáticas directas con variables
dividendo = 13
divisor = 4
resultado_division_entera = dividendo // divisor
resto_sobrante = dividendo % divisor
print(resultado_division_entera) # Muestra: 3
print(resto_sobrante) # Muestra: 1
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: particionamiento_disco.py
# Propósito: Evaluar el espacio libre sobrante de una cabina tras
realizar backups
capacidad_total_bloques = 175
bloques_usados_por_backup = 30
# Calcular cuántos backups exactos caben y cuánto espacio de bloques
sobra
backups_posibles = capacidad_total_bloques //
bloques_usados_por_backup
bloques_sobrantes = capacidad_total_bloques %
bloques_usados_por_backup
print("backups que se pueden almacenar en la cabina:")
print(backups_posibles) # Muestra: 5
print("Bloques de almacenamiento que quedan vacíos al final:")
print(bloques_sobrantes) # Muestra: 25 (el resto de la división)
```

3.3. Funciones Numéricas Predefinidas

Python tiene funciones cargadas en su núcleo para manipular datos numéricos directamente:

- `abs(x)`: Devuelve el valor absoluto de un número (su magnitud sin signo).
- `pow(x, y)`: Eleva el número base `x` al exponente `y`.
- `round(x, [y])`: Redondea un número real `x` al número de decimales indicado en `y`.
- `bin(x)`: Convierte un número entero a su representación binaria.
- `hex(x)`: Convierte un número entero a su representación hexadecimal.
- `divmod(x, y)`: Devuelve la división entera y el módulo de manera conjunta.

A) Ejemplo Básico:

```
# Redondeo simple y valor absoluto
numero_negativo = -15.678
print(abs(numero_negativo)) # Muestra: 15.678
print(round(numero_negativo, 2)) # Muestra: -15.68
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: analisis_red.py
# Propósito: Convertir parámetros de red decimales a binario/hex para
análisis manual
primer_octeto_ip = 192
latencia_ping_ms = 45.728
print("Octeto IP convertido a Binario:")
print(bin(primer_octeto_ip)) # Muestra: '0b11000000'
print("Octeto IP convertido a Hexadecimal:")
print(hex(primer_octeto_ip)) # Muestra: '0xc0'
print("Latencia media de la red redondeada a un único decimal:")
print(round(latencia_ping_ms, 1)) # Muestra: 45.7
```

3.4. Datos Booleanos y Operaciones Lógicas/Relacionales

Un **booleano** (bool) es un tipo de dato que solo puede tomar uno de dos estados posibles: True (Verdadero) o False (Falsos). Son fundamentales para definir condiciones de seguridad y saltos de flujo lógico.

- **Operadores Relacionales:** == (Igual a), != (Distinto de), <, >, <=, >=.
- **Operadores Lógicos:** and (Y lógico), or (O lógico), not (Negación).

A) Ejemplo Básico:

```
# Comparación lógica de variables numéricas
limite_edad = 18
edad_usuario = 16
puede_entrar = edad_usuario >= limite_edad
print(puede_entrar) # Muestra: False
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: auditoria_firewall.py
# Comprobamos si un puerto solicitado por una aplicación es
seguro
```

```
puerto_solicitado = 80
puerto_ssh = 22
puerto_https = 443
```

3.5. Datos de Texto: Cadenas de Caracteres (strings)

Las cadenas de texto se identifican por ir encerradas entre comillas simples (') o dobles (").

- **Operador + (Concatenación):** Permite unir dos o más cadenas de texto para formar una sola.
- **Operador * (Repetición):** Permite repetir una cadena un número determinado de veces.

A) Ejemplo Básico:

```
# Concatenación simple de palabras
saludo = "Hola"
destino = "Mundo"
mensaje_completo = saludo + " " + destino
print(mensaje_completo) # Muestra: Hola Mundo
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: constructor_rutas.py
# Propósito: Generar automáticamente la ruta de volcado de logs para
una máquina virtual
directorio_base_linux = "/var/log/nginx"
nombre_archivo_log = "error.log"
# Generación automática de la ruta completa usando concatenación
ruta_completa_sistema = directorio_base_linux + "/" +
nombre_archivo_log
print("El archivo de registro se guardará en:")
print(ruta_completa_sistema) # Muestra: /var/log/nginx/error.log
```

4. ENTRADA Y SALIDA ESTÁNDAR

4.1. Captura de Datos con input() y Conversión de Tipos (Casteo)

La función input(mensaje) detiene la ejecución del programa y espera a que el operador escriba una entrada por teclado y pulse Intro.

REGLA DE ORO DE INPUT: La función input() siempre devuelve la información capturada en forma de texto (tipo str).

Si necesitas realizar operaciones numéricas, debes transformarlo explícitamente utilizando funciones de conversión de tipos (también conocidas como funciones de casteo):

- `int(variable)`: Convierte un texto numérico compatible a entero (ej. "22" a 22).
- `float(variable)`: Convierte un texto numérico compatible a decimal (ej. "12.5" a 12.5).
- `str(variable)`: Convierte un valor numérico a texto.

A) Ejemplo Básico:

```
# Lectura de un número y cálculo de su cuadrado
numero_texto = input("Introduce un número entero: ")
numero_entero = int(numero_texto) # Casteo a entero obligatorio
resultado_cuadrado = numero_entero ** 2
print(resultado_cuadrado)
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: aprovisionamiento_cuotas.py
# Propósito: Calcular el espacio total en disco requerido para dar de
alta varios usuarios
cuota_por_usuario_gb = 15
usuarios_nuevos_texto = input("Introduce la cantidad de usuarios que
se darán de alta en el sistema: ")
usuarios_nuevos = int(usuarios_nuevos_texto) # Casteo a entero para
poder multiplicar
espacio_total_necesario_gb = usuarios_nuevos *
cuota_por_usuario_gb
print("Espacio total aproximado que se debe reservar en la partición
(GB):")
print(espacio_total_necesario_gb)
```

4.2. Visualización con `print()` y Formateo Básico

La función `print()` envía información al canal de salida estándar (la terminal). Podemos formatear o estructurar la salida mediante comas (,) o concatenando con el signo + (lo que requiere usar `str()` en las partes numéricas).

A) Ejemplo Básico:

```
# Diferentes formas de mostrar variables juntas
anio = 2026
print("El año actual es", anio)
print("El año actual es " + str(anio))
```


B) Ejemplo Práctico (Perfil ASIR):

```
# Script: telemetria_servidores.py
temperatura_cpd_grados = 21.5
cantidad_servidores_activos = 8
# Formateo de salida combinando cadenas y valores de
monitorización
print("Estado del CPD: Servidores encendidos:",
cantidad_servidores_activos, "| Temperatura del CPD:",
temperatura_cpd_grados, "grados Celsius.")
```

5. ESTRUCTURAS DE CONTROL DE FLUJO LÓGICO

5.1. La Regla Sintáctica de la Indentación

A diferencia de otros lenguajes de programación como C, Java o PHP (que delimitan los bloques internos de control mediante el uso de llaves { }), **Python utiliza la indentación (el sangrado o espacios en blanco)** como un requisito de sintaxis obligatorio.

Todas las instrucciones de un bloque de código común deben estar perfectamente alineadas a la derecha con el mismo número de espacios (la norma internacional de estilo fija de forma preferente **4 espacios en blanco**).

```
if condicion_analizada:
    instruccion_dentro_del_bloque_1
    instruccion_dentro_del_bloque_2
instruccion_fuera_del_bloque # Al volver al margen inicial, termina
el bloque anterior
```

5.2. Alternativa Condicional (if - elif - else)

Permite desviar la ejecución secuencial del script. Evalúa condiciones booleanas de forma jerárquica y secuencial. En el momento en que una condición es verdadera, ejecuta el bloque de instrucciones indentado correspondiente y salta al final de la estructura de control condicional.

A) Ejemplo Básico:

```
# Determinación simple de rangos numéricos
nota = int(input("Introduce tu nota del examen (1-10): "))
if nota >= 5:
    print("Has aprobado el examen.")
else:
    print("Has suspendido el examen.")
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: analizador_http.py
# Propósito: Evaluar el comportamiento de un servidor web según el
# código de respuesta devuelto
codigo_estado_http = int(input("Introduce el código de estado HTTP
recibido del servidor web: "))
if codigo_estado_http == 200:
    print("Diagnóstico: El servidor web responde correctamente
(OK).")
elif codigo_estado_http == 301 or codigo_estado_http == 302:
    print("Diagnóstico: El recurso solicitado está configurado como una
Redirección.")
elif codigo_estado_http == 404:
    print("Diagnóstico: Error del cliente. Recurso no encontrado en el
servidor.")
elif codigo_estado_http == 500:
    print("Diagnóstico: Error crítico interno del servidor web.")
else:
    print("Diagnóstico: Código de respuesta HTTP no reconocido o
alternativo.")
```

5.3. Bucle Iterativo Condicional (while) y Controladores (break)

El bucle while evalúa una condición booleana y, **mientras esta condición se mantenga verdadera (True)**, repite indefinidamente la ejecución del bloque de código indentado a su derecha.

- **Instrucción break:** Permite interrumpir de forma abrupta e inmediata la ejecución del bucle, saltando directamente a la primera línea de código que se encuentre fuera del bloque repetitivo.

A) Ejemplo Básico:

```
# Un bucle que cuenta de forma progresiva del 1 al 5
contador = 1
while contador <= 5:
    print(contador)
    contador = contador + 1 # Incremento básico de la variable de
control
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: shell_segura.py
# Propósito: Simular la solicitud persistente de una credencial para
acceder a la terminal de red
contrasena_correcta = "smr2023"
intentos_restantes = 3
print("--- PANEL DE SEGURIDAD DE ACCESO SSH ---")
while intentos_restantes > 0:
    clave_usuario = input("Introduce la contraseña de administración del
switch: ")
    if clave_usuario == contrasena_correcta:
        print("Acceso concedido. Cargando sesión shell...")
        break # Interrumpe y sale del bucle 'while' inmediatamente al tener
éxito
    else:
        intentos_restantes = intentos_restantes - 1
        print("Contraseña incorrecta.")
        print("Intentos de acceso restantes:", intentos_restantes)
        if intentos_restantes == 0:
            print("SISTEMA BLOQUEADO: Se ha superado el número máximo de intentos
permitidos.")
```